

≡ Hide menu

- Introduction to Vector Fields
- 🎥

Video: Project 2: Vector Fields16 min
- 📖

Reading: Textbook: Vector Fields10 min
- 🎥

Video: Vector Data From Perlin Noise8 min
- 🎥

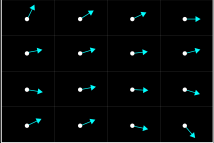
Video: Calibrating the Vector Field11 min
- Painting a Vector Field
- Neighboring Vectors and Particles

Textbook: Vector Fields

Vector fields are a fundamental concept in physics and computer graphics, particularly useful in simulating complex systems like particle movements and fluid dynamics.

VECTOR FIELD

A vector field is a mathematical construct that assigns a vector to every point in space, usually a grid. Vector fields are typically used for the simulation of fluid dynamics, electromagnetic fields, motion of particles and terrain and environment modeling.



Vector Field

What is a Vector Field?

A vector field is a map that assigns a vector to every point in a space. These vectors represent quantities that have both magnitude and direction. In a two-dimensional space, a vector field is typically visualized as a grid of arrows, each pointing in the direction of the vector at that point and with a length proportional to the vector's magnitude.

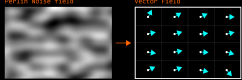
Here is a basic implementation of a Vector Field in Processing Python:

```
1 # Set up the Processing sketch
2 def setup():
3     size(400, 400) # Define the size of the canvas
4
5 # Called repeatedly to draw the sketch
6 def draw():
7     background(255) # Set the background color to white
8     # Loop through a grid of points on the canvas
9     for y in range(0, height, 20):
10        for x in range(0, width, 20):
11            # Calculate an angle using Perlin noise for smooth variation
12            angle = noise(x * 0.01, y * 0.01) * TWO_PI
13            # Create a vector pointing in the direction of the angle
14            v = PVector(cos(angle), sin(angle))
15            # Draw the vector as an arrow at point (x, y)
16            drawVector(v, x, y)
17
18 # Function to draw a vector as a line with an arrowhead
19 def drawVector(v, x, y):
20     pushMatrix() # Save the current transformation matrix
21     translate(x, y) # Move the origin to (x, y)
22     rotate(v.heading()) # Rotate the canvas to align with the vector's direction
23     line(0, 0, 10, 0) # Draw a line representing the vector
24     popMatrix() # Restore the original transformation matrix
25
```

In this example, each vector's direction is determined by **Perlin noise**, resulting in a smoothly varying field.

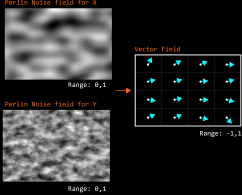
VECTORS FROM NOISE

Using Perlin noise in this way produces a smoothly varying vector field with organic changes in direction, making it useful for simulations like fluid dynamics or particle movement in a natural environment.



VECTORS FROM NOISE

To create a vector field from adding two Perlin noise values, you can utilize the noise function to derive the x and y (or z for 3D) components of each vector in your field.



We will be constructing a Vector Field Class that will be able to interact with the Particle System we developed in Week 1. The Class below also shows how we can animate the Perlin Noise and see the results in the Vector Field.

```
1 from Force import Force
2 import random
3
4 class Vector_Field:
5     # Constructor for the Vector_Field class
6     def __init__(self, cols, rows, world_x, world_y):
7         self.cols = cols # Number of columns in the vector field
8         self.rows = rows # Number of rows in the vector field
9         self.all_forces = [] # List to store all force vectors
10        self.world_x = world_x # Width of the vector field area
11        self.world_y = world_y # Height of the vector field area
12        # Calculate the size of each cell in the grid
13        self.cell_size_x = float(world_x) / cols
14        self.cell_size_y = float(world_y) / rows
15        self.max_vec_size = 17 # Maximum size of the vector
16        # Print cell sizes for verification
17        println(self.cell_size_x)
18        println(self.cell_size_y)
19        # Phases for animating the vector field
20        self.phase_01 = 0.1
21        self.phase_02 = 0.3
22
23 # Initialize the vector field with forces
24 def initiate_vecfield(self):
25     for i in range(self.cols):
26         for j in range(self.rows):
27             # Calculate the position of each force in the grid
28             pos = PVector(i * self.cell_size_x + self.cell_size_x / 2,
29                           j * self.cell_size_y + self.cell_size_y / 2)
30
31             # Generate noise values and map them to vector sizes
32             n = noise(i * 0.1, j * 0.1)
33             n2 = noise(i * 0.04, j * 0.04)
34             n = map(n, 0, 1, -self.max_vec_size, self.max_vec_size)
35             n2 = map(n2, 0, 1, -self.max_vec_size, self.max_vec_size)
36
37             # Create a new force and add it to the list
38             new_force = Force(pos, PVector(n, n2))
39             self.all_forces.append(new_force)
40
```

- The **Vector_Field** class creates and manages a field of vectors (**forces**).
- In **initiate_vecfield**, it initializes the field with forces based on **Perlin noise**.
- In **animate_vecfield**, it updates the field with new forces, creating an animated effect. The phases (phase_01 and phase_02) are incremented to change the noise values over time, giving a dynamic appearance.
- The **run** method is the main function that clears the current forces, animates new ones, and displays them.
- The display method iterates through all forces and runs them, which involves drawing them on the canvas.

Go to next item ✓ Completed

